

How PCA And NCA Uncover The Hidden Structure In High-dimensional Data

Abstract

Classifying high-dimensional data is a fundamental problem with numerous real-world applications. These large sets are often difficult to interpret and visualize. Here, we explore two algorithms that reduce the dimension of data without losing the ability to accurately predict values of new unseen data points. We study the mathematical foundations of Principal Component Analysis (PCA), essentially an application of linear algebra, and Neighborhood Components Analysis (NCA), a more modern approach framed as an optimization problem.

Principal Component Analysis (PCA)

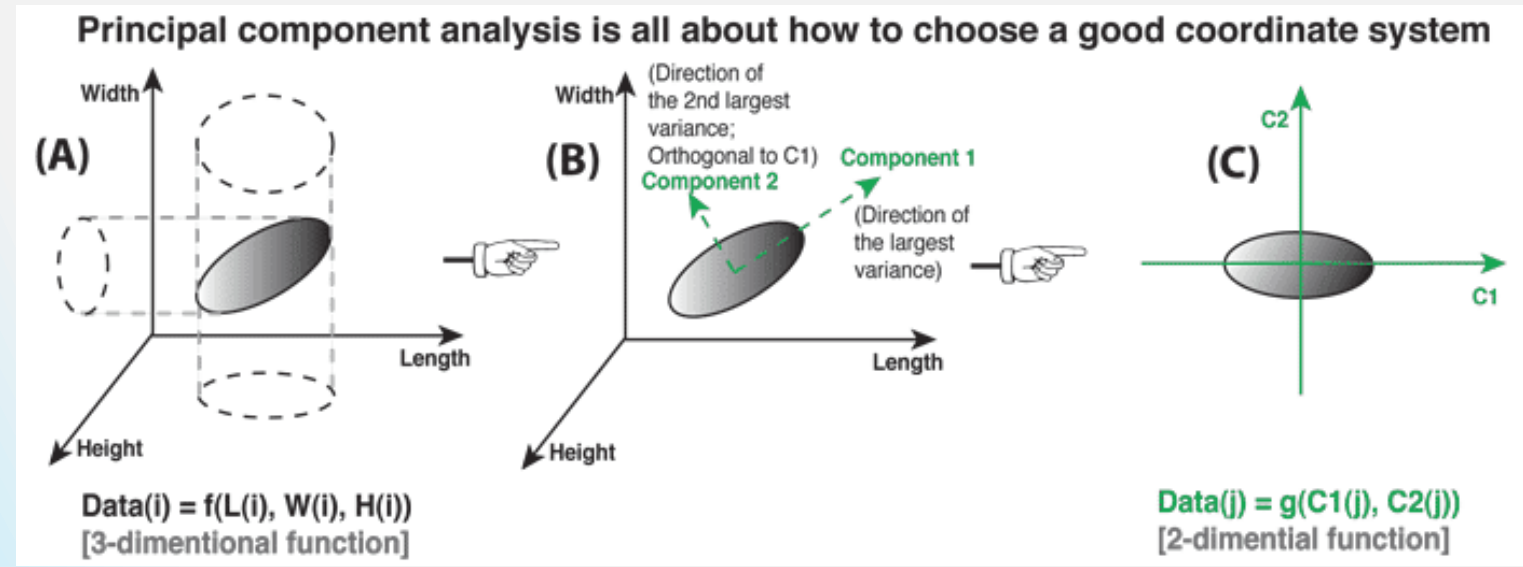
Imagine our data as a *cloud* of points in R^n . PCA is a *change of basis*. We want to find a new *orthonormal basis* $\{v_1, v_2, \dots, v_n\}$ such that the first basis vector points in the direction of the **greatest “spread”** of the data.

Organize the results into the $n \times d$ matrix, $X = [X_1 | X_2 | \dots | X_d]$. Where each X_j is an n – dimensional column vector representing features.

We transform our data matrix X into $Z = X - \mu$ to translates the **“center of mass” of our data cloud to the origin**. This ensures that the variance is calculated relative to the actual center of the data.

The Covariance Matrix: $S = \frac{1}{n-1} Z^T Z$

This describes the data’s shape. The diagonal entries are the **variances** (spread along axes), and the off-diagonal entries are **covariances** (how features change together).



Geometrically, S defines a **hyper-ellipsoid (an n - dimensional egg)**. PCA finds the principal axes of this egg.

We solve the characteristic equation $\det(S - \lambda I) = 0$ to find eigenvalues λ_i and eigenvectors v_i . Since S is a **symmetric matrix**, the Spectral Theorem guarantees that our eigenvectors (Principal Components) are orthogonal (perpendicular). The eigenvalue λ_i tells us exactly how much **“spread”** exists along the axis v_i .

To reduce our $n \times d$ dataset, we create a **Projection Matrix P** using only the first eigenvector for 1D reduction, first 2 eigenvectors for 2D reduction and so on ($P = [v_1]$ for 1D and $P = [v_1, v_2]$ for 2D).

The Transformation: We multiply our centered data by this matrix: $Z_{new} = Z \cdot P$.

Neighborhood Component Analysis (NCA)

To understand NCA, we must start with K-Nearest Neighbors (KNN), where a point is classified by the **majority vote of k closest neighbors**. The “closeness” is usually defined by the Euclidean distance, $d(x_i, x_j) = ||x_i - x_j||$. However, in high dimensions, standard Euclidean distance is often misleading. Some pixels matter more than others, but standard KNN treats all dimensions as equally important.

We seek linear matrix A that transforms our space into a lower-dimensional one, $y = Ax$, so that KNN becomes more accurate. Under the transformation A , the distance becomes $d(x_i, x_j) = ||Ax_i - Ax_j||$.

If we try to optimize A by simply counting correct KNN predictions, we encounter a problem: as A changes, neighbors **“jump”** in and out of the set. This function is **non-differentiable**, making it impossible to use Calculus.

The **NCA Solution [Stochastic(Soft) Neighbors]**: NCA replaces the “hard” choice of neighbors with a **Softmax probability**. Instead of a point having a single neighbor, we say **every other point j has a certain probability p_{ij} of being i ’s neighbor**:

$$p_{ij} = \frac{\exp(-||Ax_i - Ax_j||^2)}{\sum_{k \neq i} \exp(-||Ax_i - Ax_k||^2)}, \quad p_{ii} = 0$$

We define our **Objective Function $f(A)$** as the the sum of probabilities that *each point i selects a*

neighbor j from its own class (C_i):

$$f(A) = \sum_i \sum_{j \in C_i} p_{ij} = \sum_i p_i$$

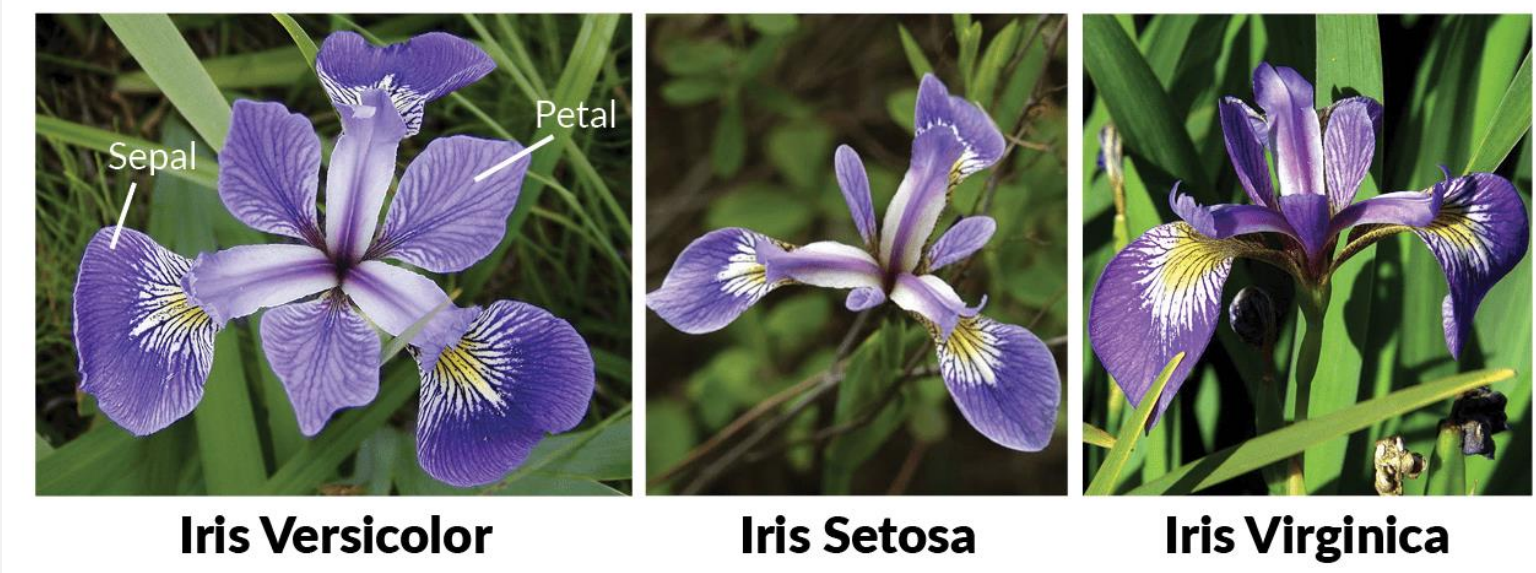
Since A is smooth, we use Gradient Descent to iteratively update A :

$$A_{next} = A_{curr} + \gamma \nabla f(A)$$

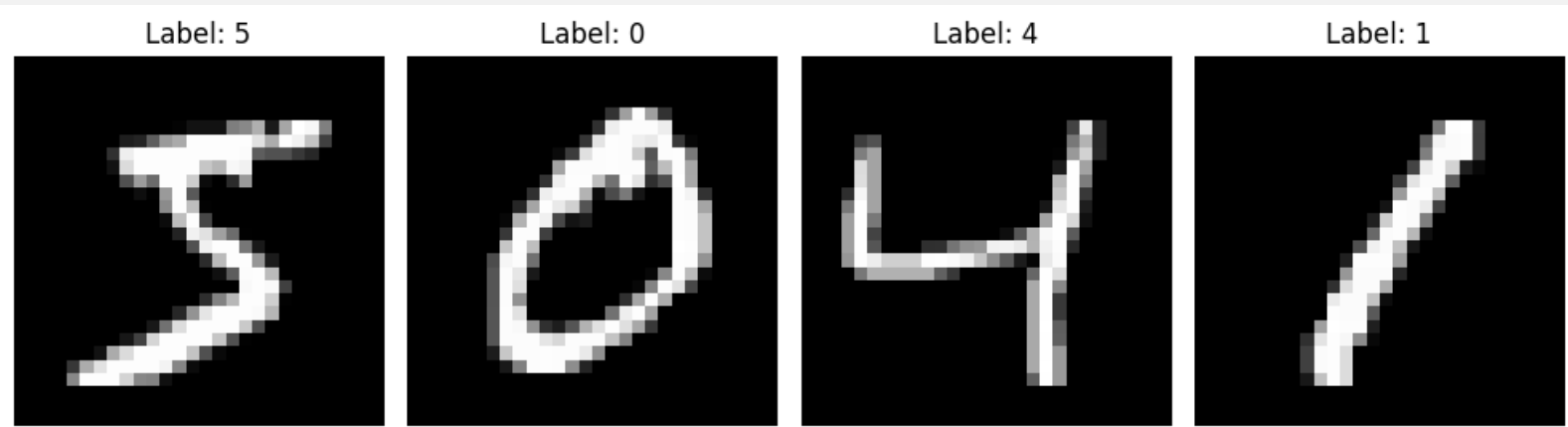
By defining A as a $k \times n$ matrix, the algorithm learns a projection that compresses high-dimensional data into **k best coordinates** for cluster separation.

Datasets

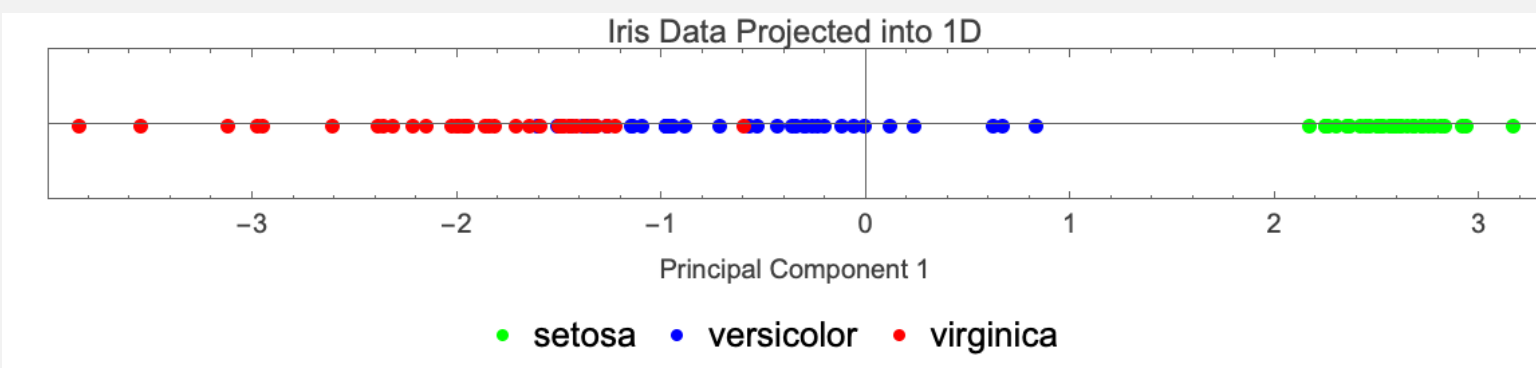
The **Fisher’s Iris Dataset** consists of 150 samples across 3 species (Setosa, Versicolor, Virginica). Each sample has 4 features: sepal and petal length/width:



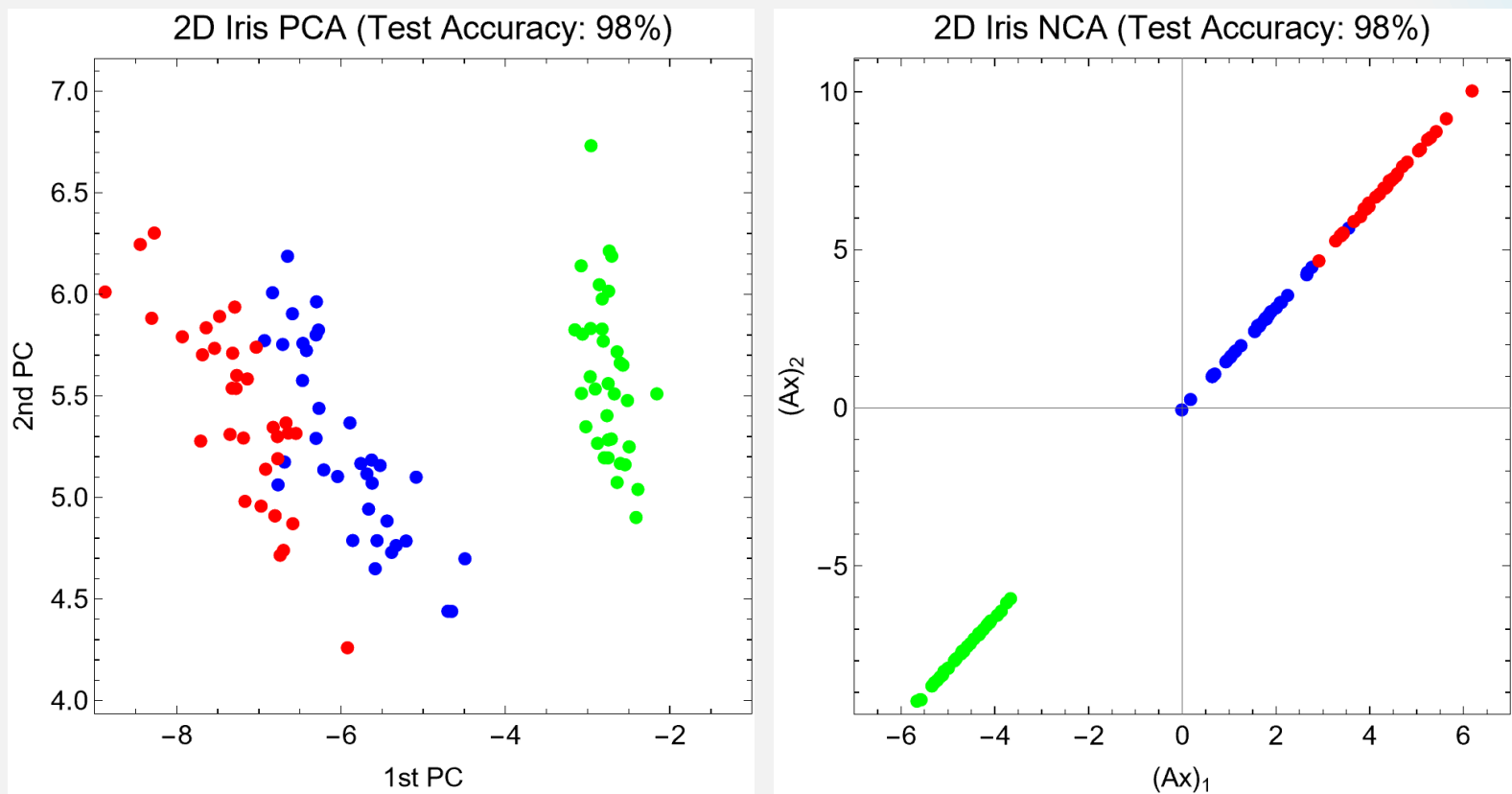
Digits Dataset: A collection of 1,797 handwritten digits (0–9). Each digit is an 8x8 grayscale image, resulting in 64 dimensions:



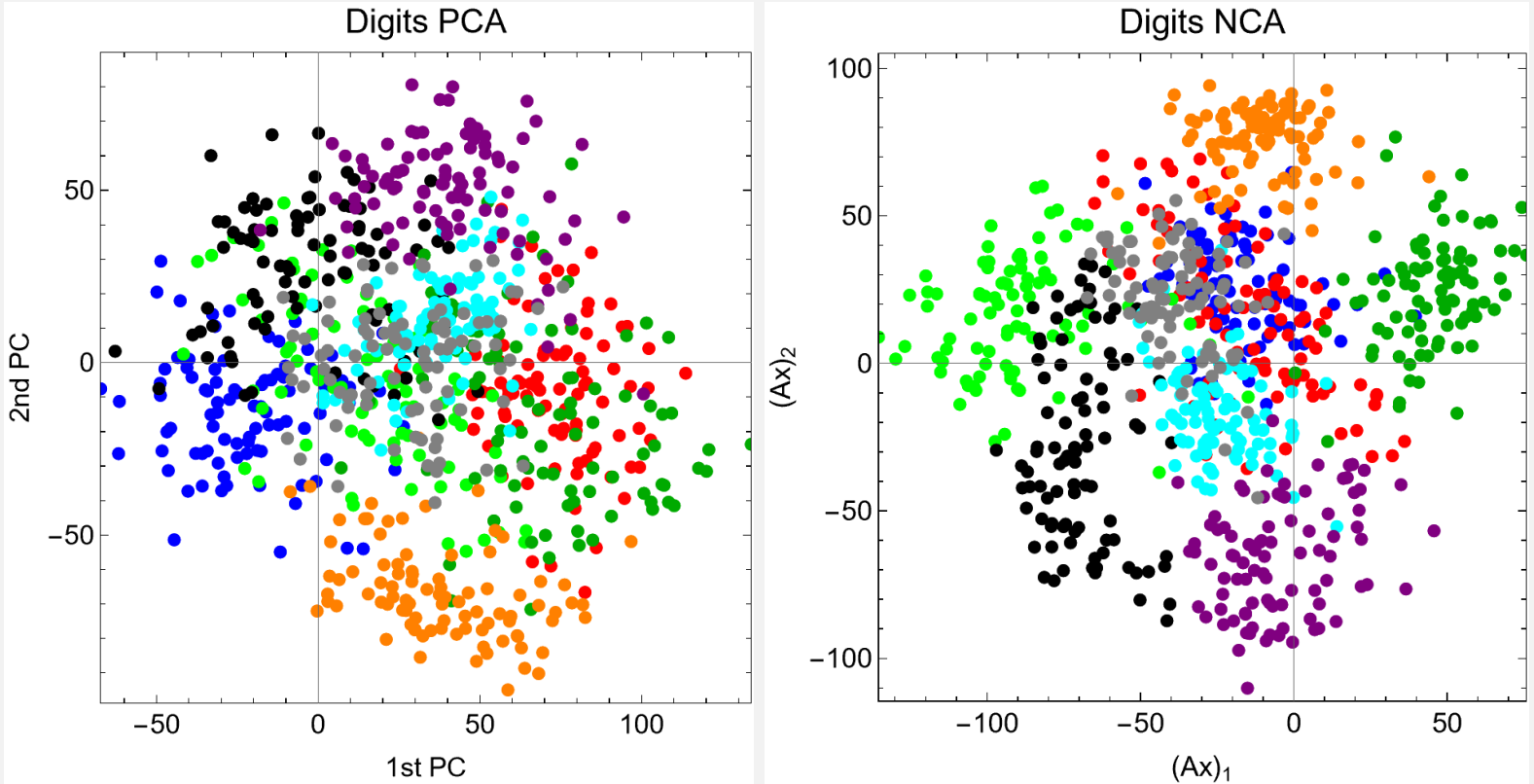
Results



While a naive 1D projection results in significant overlap between *Versicolor* and *Virginica*, the 2D PCA projection reveals a clear linear boundary. The transformation from $R^4 \rightarrow R^2$ preserves nearly all critical information.



We achieved an accuracy of 98% confirming that the species’ unique signatures are preserved even after discarding 50% of the dimensions.



For the Digits’, PCA distinct numbers like 0, 1, and 4 separate into clear clusters, while structurally similar digits remain closer together. NCA reorganizes data to emphasize differences between classes. It minimizes neighbor errors, resulting in tighter groupings and improved separation.

Algorithm	Method	Cost	Goal
PCA	Linear Algebra	Low	Max Variance
NCA	Optimization	High	Max Class Separation

More data does not always mean better understanding. PCA reveals the natural structure of data, while NCA optimizes it for prediction. Together, they show that dimensionality reduction uncovers the fundamental geometry of information.

References

Hofstrand, A. (2026). *MATH 330: Computational Analysis Course Notes*. NYIT.

Jacob Goldberger, Sam Roweis, Geoff Hinton, Ruslan Salakhutdinov. *Advances in Neural Information Processing Systems 17 (NIPS 2004)*.

Meng, C. (2013). *An Intuitive Explanation of PCA*. Meng's Notes. [mengnote.blogspot.com]